

UFU - FACOM - BCC
Algoritmos e Estruturas de Dados - II

Implementação de um sistema de manipulação de grafos

Integrantes do Grupo

- Matheus da Silva Carolino
- Matheus Henrique Ribeiro Cardoso
- Pedro Tavares Oliveira
- Samuel
- Santiago Vitor Santos do Carmo
- Simão Baptista Sunga

Instruções para uso

1. **Clonar o repositório git:** Abra o terminal na pasta onde deseja salvar o projeto e rode o comando de clone usando o [link html do repositório](#).
2. **Abra na sua IDE de preferência:** Basta abrir a pasta do projeto usando a sua IDE e executar o arquivo principal "Main.c".

Explicação da implementação

O sistema foi desenvolvido respeitando as boas práticas de modularização. Para isso, foram criados os seguintes arquivos de cabeçalho (.h): **grafo**, **algoritmos**, **estatisticas**, **menu**, **validacao** e **teste**, bem como os respectivos arquivos de implementação (.c). A integração de toda a infraestrutura do projeto foi realizada no arquivo principal (main.c).

Conforme exigido nos requisitos técnicos, a entrada de dados ocorre por meio de arquivos (.txt) armazenados no diretório (data/). Já a interação com o usuário é feita via terminal através de um menu interativo (menu.c). A implementação do menu utiliza uma estrutura *switch-case*, na qual cada opção aciona funções específicas do sistema.

A seguir, apresenta-se a descrição do arquivo **grafo.c** e de sua respectiva implementação no projeto.

1. (grafo.c): Responsável pela manipulação geral dos grafos dentro do sistema. Este módulo concentra as funções auxiliares para criação, alocação dinâmica de memória, liberação do grafo e de suas estruturas, exibição, inserção ordenada de arestas e leitura de dados via arquivo.
 - a. (Grafo* criarGrafo): Inicializa o grafo vazio utilizando a abordagem de lista de adjacência. A função define o número de vértices e arestas (com base na leitura do arquivo) e identifica se o grafo é direcionado ou não, utilizando alocação dinâmica de memória para gerenciar essas estruturas.
 - b. (void liberarGrafo): Gerencia a liberação do grafo e de sua lista da memória. Esta função é fundamental para evitar o desperdício de recursos de hardware durante a execução do sistema (memory leak).
 - c. (void mostrarGrafo): Exibe o grafo no terminal imprimindo sua lista de adjacência. Caso um vértice não possua vizinhos o sistema exibe a mensagem “sem conexões”.
 - d. (void inserirOrdenado): Insere as arestas de forma ordenada na lista. A lógica abrange dois cenários: a inserção no início (quando a lista está vazia ou o elemento possui o menor valor) e a inserção no meio ou no fim da lista.
 - e. (void adicionarAresta): Adiciona uma aresta conectando um vértice de origem a um de destino, utilizando a função auxiliar **inserirOrdenado**. Ela verifica se o grafo é direcionado ou não para aplicar o tratamento correto de inserção em ambos os casos.
 - f. (GrafoStatus lerGrafoDeArquivo): Responsável por abrir, ler os dados, preencher a estrutura do grafo e fechar o arquivo. Caso ocorra alguma falha no processo, a função retorna um **status** de erro para que o sistema exiba a mensagem apropriada ao usuário, caso contrário, confirma com sucesso o carregamento de dados.

A seguir, apresenta-se a descrição do arquivo **algoritmos.c** e de sua respectiva implementação no projeto.

1. (algoritmos.c): Este módulo concentra as implementações dos algoritmos de buscas, ordenação, árvores geradoras, caminhos mínimos e análise estrutural de componentes em grafos.
 - a. (GrafoStatus DFSRecurativa): Executa uma busca em profundidade a partir de um vértice de origem utilizando uma abordagem recursiva. Durante a execução, visita recursivamente os vértices alcançáveis e exibe a ordem de visitação.
 - b. (GrafoStatus DFSIterativa): Executa uma busca em profundidade a partir de um vértice de origem utilizando uma pilha estática explícita para simular a recursão. Exibe a ordem de visitação dos vértices alcançados.
 - c. (GrafoStatus DFSCompConexos): Identifica os componentes conexos de um grafo não direcionado por meio de buscas em profundidade. Para cada vértice ainda não visitado, inicia uma nova DFS, identificando um componente conexo distinto. Ao final, exibe os vértices pertencentes a cada componente.
 - d. (GrafoStatus BFS): Executa a busca em largura utilizando uma fila estática para navegação por níveis. Exibe a ordem de visitação dos vértices e calcula e imprime as distâncias em número de arestas da origem para os demais vértices.
 - e. (GrafoStatus ordemTopologica): Realiza a ordenação topológica de um grafo acíclico dirigido baseado no Algoritmo de Kahn. Utiliza um vetor para mapear os graus de entrada e uma fila com inserção descendente para desempate prioritário. A função detecta e retorna um **status** de erro caso o grafo seja cíclico, o que impede a existência de uma ordem topológica.
 - f. (GrafoStatus algoritmoDijkstra): Determina os caminhos mínimos de um vértice de origem aos demais, utilizando relaxamento de arestas e a reconstrução completa do caminho percorrido de trás para frente usando uma pilha auxiliar para, em seguida, exibir as distâncias.
 - g. (GrafoStatus caminhoCritico): A função primeiro verifica se o grafo passado como argumento é um grafo direcionado acíclico, a função retorna ERRO_GRAFO_COM_CICLOS caso o grafo possui um ciclo. Após isso, ela calcula o valor de

cada aresta de um caminho presente no grafo e imprime a maior distância. Por fim ela imprime o caminho que foi usado para calcular a maior distância.

- h. (GrafoStatus algoritmoKosaraju): Primeiro a função faz uma DFS e armazena todos os vértices do grafo passado como argumento em uma pilha, onde o primeiro vértice que termina a DFS é o topo da pilha e o último é o início da pilha. Após isso, é criado um novo grafo que é transposto em relação ao grafo original. Por fim, ela pega todos os vértices que terminam por último da DFS e mostram os componentes fortemente conexos deles aplicando uma DFS no grafo transposto.

A seguir, apresenta-se a descrição do arquivo **estatistica.c** e de sua respectiva implementação no projeto.

1. (estatistica.c): Este módulo é responsável por calcular e exibir as estatísticas do grafo, incluindo a presença de ciclos, o grau dos vértices, a conectividade, a direção do grafo (direcionado ou não direcionado) e sua densidade.
 - a. (bool temCiclo): Verifica se um grafo possui ciclos utilizando uma busca em profundidade com coloração de vértices. Emprega uma versão adaptada da DFS para tratar corretamente grafos direcionados e não direcionados, retornando um valor booleano indicando a existência de ciclos.
 - b. (bool isConexo): Verifica se um grafo não direcionado é conexo utilizando uma busca em profundidade. Executa uma DFS no grafo e verifica se todos os vértices foram alcançados. Caso contrário, o grafo é considerado desconexo.
 - c. (bool isFortementeConexo): Verifica se um grafo direcionado é fortemente conexo utilizando o algoritmo de Kosaraju. Executa uma DFS no grafo original e outra no grafo transposto para verificar se todos os vértices são alcançáveis em ambos os casos. Se todas as buscas alcançarem todos os vértices, o grafo é considerado fortemente conexo.
 - d. (float calcularDensidade): Calcula a densidade do grafo com base no número de arestas e no número máximo de arestas

possíveis. A implementação é adaptada para grafos direcionados e não direcionados.

- e. (void calcularImprimirGraus): Calcula os graus dos vértices do grafo. Para grafos direcionados, determina e imprime os graus de entrada e de saída de cada vértice. Para grafos não direcionados, imprime apenas o grau de cada vértice.
- f. (void exibirEstatisticas): Reúne e exibe as principais estatísticas do grafo, integrando as funcionalidades de verificação de ciclos, conectividade, graus dos vértices, densidade e demais informações relevantes, de acordo com o tipo do grafo.

A seguir, apresenta-se a descrição do arquivo **menu.c** e de sua respectiva implementação no projeto.

- 1. Este módulo é responsável por implementar a interface de interação com o usuário, gerenciando a lógica do menu principal. Além de exibir as opções disponíveis, coordena a execução das funcionalidades do sistema, integrando os diferentes módulos do projeto.
 - a. (void exibirMenu): Exibe o menu principal do sistema e gerencia a interação com o usuário. Processa a opção selecionada, verifica se há um grafo carregado quando necessário e coordena a execução das funcionalidades disponíveis, integrando os diferentes módulos do projeto. Trata-se da principal função do módulo, utilizando funções auxiliares, como `executarBuscaEmProfundidade` e `executarBuscaEmLargura`, para encapsular funcionalidades específicas e facilitar a organização, a manutenção e a extensibilidade do código.

A seguir, apresenta-se a descrição do arquivo **validacao.c** e de sua respectiva implementação no projeto.

- 2. (validacao.c): Este módulo é responsável por centralizar as verificações de erro do sistema. Ele define os códigos de retorno utilizados pelas demais funções e permite que os algoritmos apenas

informem o resultado da execução, deixando a exibição das mensagens de erro em um único local.

- a. (void imprimirMensagemGrafo): Exibe a mensagem correspondente ao código de retorno recebido. Utiliza uma estrutura de um switch case para associar cada valor de GrafoStatus à sua respectiva mensagem.

A seguir, apresenta-se a descrição do arquivo **teste.c** e de sua respectiva implementação no projeto.

3. (teste.c): Este módulo é responsável por executar uma bateria de testes sobre o grafo carregado no sistema, medindo o tempo de execução dos principais algoritmos implementados.
 - a. (void bateriaDeTestes): Executa os algoritmos de Ordenação Topológica, Dijkstra, DFS (recursiva e iterativa), BFS, Prim, Kosaraju e Caminho Crítico, medindo o tempo de execução de cada um. Ao final, exibe os resultados obtidos, permitindo analisar o desempenho das implementações sobre o grafo carregado.

Análise de complexidade de cada algoritmo

A seguir, apresenta-se a descrição detalhada da complexidade das funções do arquivo **algoritmos.c** do projeto.

- a. (GrafoStatus DFSRecursiva): O algoritmo proposto processa o grafo com a complexidade de tempo $O(V + A)$ e sua complexidade espacial é $O(V)$ exigindo um espaço auxiliar temporário de complexidade $O(V)$.
- b. (GrafoStatus DFSIterativa): O algoritmo tem as mesmas complexidade da abordagem recursiva: Tempo $O(V + A)$ e Espaço $O(V)$.
- c. (GrafoStatus DFSCompConexos): O algoritmo também tem as mesmas complexidade das abordagens anteriores: Tempo $O(V + A)$ e Espaço $O(V)$.

- d. (GrafoStatus BFS): O algoritmo proposto processa o grafo com complexidade de tempo $O(V + A)$ e sua complexidade espacial é $O(V)$ exigindo um espaço auxiliar temporário de complexidade $O(V)$.
- e. (GrafoStatus ordemTopologica): O algoritmo proposto processa o grafo com complexidade de tempo $O(V^2 + A)$ e sua complexidade espacial é $O(V)$ exigindo um espaço auxiliar temporário de complexidade $O(V)$.
- f. (GrafoStatus algoritmoDijkstra): O algoritmo proposto processa o grafo com complexidade de tempo $O(V^2)$ e sua complexidade espacial é $O(V)$ exigindo um espaço auxiliar temporário de complexidade $O(V)$.
- g. (GrafoStatus caminhoCritico): O algoritmo proposto processa o grafo com a complexidade de tempo $O(V + A)$ e o de tempo $O(V + A)$.
- h. (GrafoStatus algoritmoKosaraju): O algoritmo proposto, levando em conta a representação usada (lista de adjacência), tem as complexidades: Tempo $O(V + A)$ e Espaço $O(V + A)$.

Resultados obtidos para cada arquivo de teste

A seguir, serão apresentados os resultados obtidos ao executar as funções dos arquivos **algoritmos.c** e **estatisticas.c** com a utilização dos arquivos de testes descritos no moodle.

Arquivo 1: grafo1.txt (Grafo não-direcionado simples)

4 5

0 1 2

0 2 4

1 2 1

1 3 3

2 3 3

=== DFS Recursiva ===

0 -> 1 -> 2 -> 3

=== DFS Iterativa ===

0 -> 1 -> 2 -> 3

Retorno da execução da função BFS

Erro: Grafo com pesos invalidos

Retorno da execução da função ordemTopologica

Erro: O grafo carregado é não direcionado e não pode ser usado nesse algoritmo!

=== ÁRVORE GERADORA MÍNIMA (PRIM) ===

Aresta: 0 - 1 | Peso: 2

Aresta: 1 - 2 | Peso: 1

Aresta: 1 - 3 | Peso: 3

=====

Peso Total da AGM: 6

=====

=== DIJKSTRA (Origem: 0) ===

Vertice | Distancia | Caminho

-----|-----|-----

0 | 0 | 0

1 | 2 | 0 -> 1

2 | 3 | 0 -> 1 -> 2

3 | 5 | 0 -> 1 -> 3

=== COMPONENTES CONEXOS (DFS ITERATIVA) ===

Componente 1: 0 -> 1 -> 2 -> 3

Retorno da execução da função caminhoCritico

Erro: O grafo carregado é não direcionado e não pode ser usado nesse algoritmo!

=== ESTATISTICAS DO GRAFO ===

- Numero de vertices: 4

- Numero de arestas: 5

- Tipo de grafo: Nao-direcionado

- Grau de cada vertice:

Vertice 0: Grau = 2

Vertice 1: Grau = 3

Vertice 2: Grau = 3

Vertice 3: Grau = 2

- Conexo: SIM

- Presenca de ciclos: SIM, contem ciclos

- Densidade do grafo: 0.8333

Arquivo 2: grafo2.txt (DAG para ordenação topológica)

6 7

0 1 1

0 2 1

1 3 1

2 3 1

2 4 1

3 5 1

4 5 1

=== BUSCA EM PROFUNDIDADE RECURSIVA (Origem 0) ===

Ordem: 0 -> 1 -> 3 -> 5 -> 2 -> 4

=== BUSCA EM PROFUNDIDADE ITERATIVA (Origem 0) ===

Ordem: 0 -> 1 -> 3 -> 5 -> 2 -> 4

=== BUSCA EM LARGURA (Origem: 0) ===

Ordem: 0 -> 1 -> 2 -> 3 -> 4 -> 5

=== ORDENACAO TOPOLOGICA ===

Ordem: 0 -> 2 -> 4 -> 1 -> 3 -> 5

Retorno da execução da função AlgoritmoPrim

Erro: O grafo carregado é direcionado e não pode ser usado nesse algoritmo!

=== DIJKSTRA (Origem: 0) ===

Vertice | Distancia | Caminho

-----|-----|-----

0 | 0 | 0

1 | 1 | 0 -> 1

2 | 1 | 0 -> 2

3 | 2 | 0 -> 1 -> 3

4 | 2 | 0 -> 2 -> 4

5 | 3 | 0 -> 1 -> 3 -> 5

=== COMPONENTES FORTEMENTE CONEXOS (KOSARAJU) ===

Componente de 1: 0

Componente de 2: 2

Componente de 3: 4

Componente de 4: 1

Componente de 5: 3

Componente de 6: 5

=== CAMINHO CRITICO ===

A maior distancia que pode ser percorrida nesse DAG é: 3

Caminho Critico: 0 -(1)-> 1 -(1)-> 3 -(1)-> 5

=== ESTATISTICAS DO GRAFO ===

- Numero de vertices: 6
- Numero de arestas: 7
- Tipo de grafo: Direcionado (Digrafo)
- Grau de cada vertice:
 - Vertice 0: Grau de Entrada = 0 | Grau de Saida = 2
 - Vertice 1: Grau de Entrada = 1 | Grau de Saida = 1
 - Vertice 2: Grau de Entrada = 1 | Grau de Saida = 2
 - Vertice 3: Grau de Entrada = 2 | Grau de Saida = 1
 - Vertice 4: Grau de Entrada = 1 | Grau de Saida = 1
 - Vertice 5: Grau de Entrada = 2 | Grau de Saida = 0
- Fortemente conexo: NAO
- Presenca de ciclos: NAO possui ciclos (Aciclico)
- Densidade do grafo: 0.2333

Arquivo 3: grafo3.txt (Grafo com ciclo) OBS: Foi carregado como grafo direcionado

4 5

0 1 1

1 2 1

2 0 1

2 3 1

1 3 1

=== BUSCA EM PROFUNDIDADE RECURSIVA (Origem 0) ===

Ordem: 0 -> 1 -> 2 -> 3

=== BUSCA EM PROFUNDIDADE ITERATIVA (Origem 0) ===

Ordem: 0 -> 1 -> 2 -> 3

=== BUSCA EM LARGURA (Origem: 0) ===

Ordem: 0 -> 1 -> 2 -> 3

Distancias a partir do vertice 0:

Vertice 0: distancia = 0

Vertice 1: distancia = 1

Vertice 2: distancia = 2

Vertice 3: distancia = 2

=== ORDENACAO TOPOLOGICA ===

Ordem:

O grafo possui pelo menos um ciclo. A ordenacao acima esta incompleta.

Retorno da função AlgoritmoPrim

Erro: O grafo carregado é direcionado e não pode ser usado nesse algoritmo!

=== DIJKSTRA (Origem: 0) ===

Vertice | Distancia | Caminho

-----|-----|-----

0 | 0 | 0

1 | 1 | 0 -> 1

2 | 2 | 0 -> 1 -> 2

3 | 2 | 0 -> 1 -> 3

=== COMPONENTES FORTEMENTE CONEXOS (KOSARAJU) ===

Componente de 1: 0 -> 2 -> 1

Componente de 2: 3

Retorno da função caminhoCritico

Erro: O grafo contém ciclos e não pode ser usado nesse algoritmo!

=== ESTATISTICAS DO GRAFO ===

- Numero de vertices: 4

- Numero de arestas: 5

- Tipo de grafo: Direcionado (Digrafo)

- Grau de cada vertice:

Vertice 0: Grau de Entrada = 1 | Grau de Saida = 1

Vertice 1: Grau de Entrada = 1 | Grau de Saida = 2

Vertice 2: Grau de Entrada = 1 | Grau de Saida = 2

Vertice 3: Grau de Entrada = 2 | Grau de Saida = 0

- Fortemente conexo: NAO

- Presenca de ciclos: SIM, contem ciclos

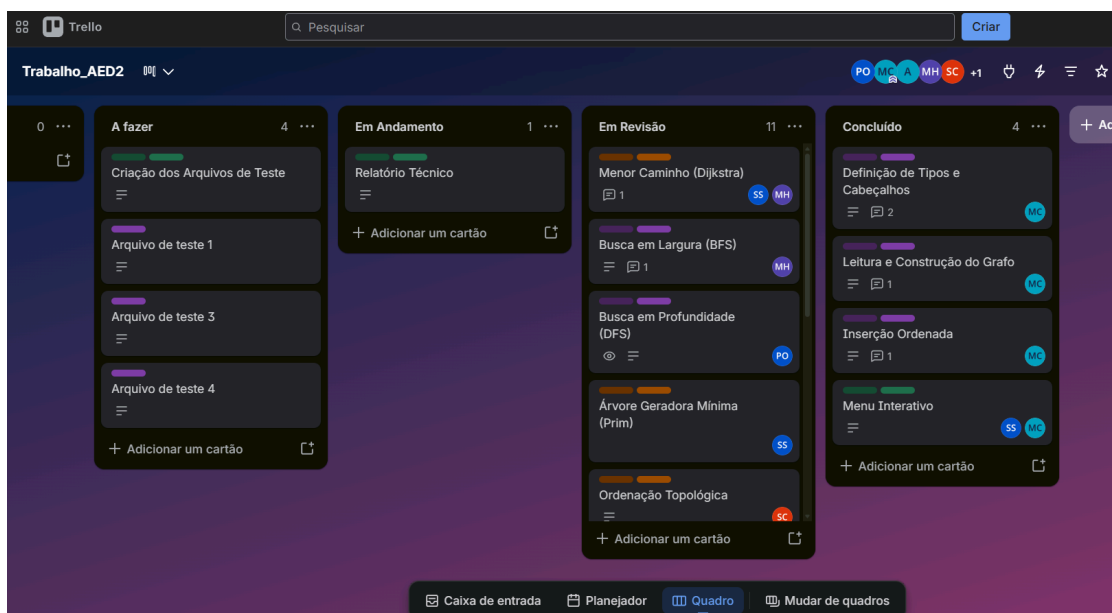
- Densidade do grafo: 0.4167

Arquivo 4: grafo4.txt (Grafo ponderado para Dijkstra) OBS: Foi carregado como grafo direcionado não direcionado

5 7
0 1 4
0 2 2
1 2 1
1 3 5
2 3 8
2 4 10
3 4 2

Dificuldades encontradas e soluções adotadas

Ao iniciar o trabalho, a principal dificuldade da equipe foi organizar a execução do projeto. Para isso, foi utilizado o Trello, criado pelo integrante Matheus Carolino, seguindo a metodologia Kanban. As atividades foram divididas em módulos, organizadas por prioridade e distribuídas entre os membros, facilitando o acompanhamento do desenvolvimento e a comunicação da equipe.



Com o avanço do projeto, surgiu a necessidade de desenvolver o código de forma colaborativa. Para isso, foi criado um repositório no GitHub (<https://github.com/Matheus-Carolino/Algoritmos-Em-Grafos.git>), permitindo que todos os membros tivessem acesso à versão atualizada do sistema e pudessem contribuir simultaneamente.

Durante a implementação, uma das principais dificuldades esteve relacionada ao tratamento de grafos direcionados e não direcionados. Como os arquivos de entrada não informavam essa característica, foi adicionado à estrutura do grafo o campo direcionado, que recebe o valor 1 para grafos direcionados e 0 para grafos não direcionados. Essa alteração permitiu que os algoritmos identificassem o tipo de grafo fossem executados corretamente.

Além disso, foi necessário estudar alguns algoritmos antes de sua implementação, principalmente aqueles que não haviam sido abordados em sala de aula. Esse processo contribuiu para uma compreensão mais adequada dos métodos utilizados e para uma implementação mais consistente.

De forma geral, as dificuldades enfrentadas contribuíram para o aprendizado da equipe e para a melhoria da organização do desenvolvimento, permitindo a conclusão do projeto de maneira colaborativa.